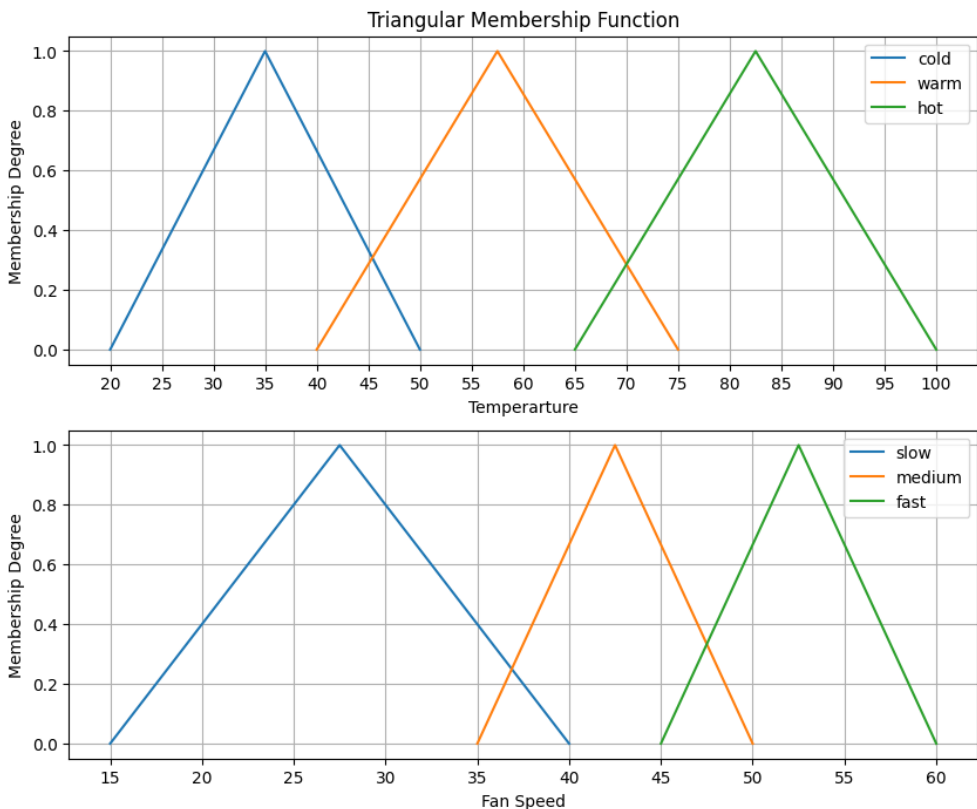


Experiment-01: Fuzzy Logic System

- Fuzzy Logic is a form of many-valued logic in which truth values of variables may be any real number between 0 and 1.
- Unlike traditional binary logic (Boolean logic), which sees the world in black and white (True/1 or False/0), fuzzy logic handles the concept of "partial truth."
- It mimics human reasoning, which relies on vague and approximate terms rather than precise mathematics.

Problem-01: Fan Speed Controller

Input	Output
temperature={ 'cold':[20,50], 'warm':[40,75], 'hot':[65,100] }	fan_speed={ 'slow':[15,40], 'medium':[35,50], 'fast':[45,60] }
#Rules: 1. If the temperature is hot, the fan will move fast. 2. If the temperature is warm, the fan will move medium. 3. If the temperature is cold, the fan will move slow.	
# Membership Graph: Triangular Membership Function  The figure consists of two line graphs. The top graph, titled 'Triangular Membership Function', plots 'Membership Degree' (y-axis, 0.0 to 1.0) against 'Temperature' (x-axis, 20 to 100). It shows three functions: 'cold' (blue line, peak at 35), 'warm' (orange line, peak at 57.5), and 'hot' (green line, peak at 82.5). The bottom graph plots 'Membership Degree' (y-axis, 0.0 to 1.0) against 'Fan Speed' (x-axis, 15 to 60). It shows three functions: 'slow' (blue line, peak at 27.5), 'medium' (orange line, peak at 42.5), and 'fast' (green line, peak at 52.5).	

Source Code:

```
def rules(temp):
    if(temp=='hot'):
        return np.sum(fan_speed['fast'])/2
    elif(temp=='warm'):
        return np.sum(fan_speed['medium'])/2
    elif(temp=='cold'):
        return np.sum(fan_speed['slow'])/2

def membership_degree(x,s):
    a=temperature[s][0] # minimum value
    b=np.mean(temperature[s]) # mid value
    c=temperature[s][1] #max value
    if x>=c:
        return 0
    if x>b:
        return (c-x)/(c-b)
    if x>a:
        return (x-a)/(b-a)
    else:
        return 0

def Fuzzy(x):
    membership_value=[]
    for i,j in temperature.items():
        result=membership_degree(x,i) # take the membership value from given input
        membership_value.append(result)
    print(membership_value)

    crisp_speed=[]
    for i in temperature:
        crisp_speed.append(rules(i)) # storing the rule
    print(crisp_speed)

    crisp_speed=np.array(crisp_speed)
    try:
        crisp_speed=crisp_speed*np.array(membership_value) #multiplication
        crisp_speed=np.sum(crisp_speed)/np.sum(membership_value) # Divide
    except:
        crisp_speed=0
    return crisp_speed

x=45
print(f'{Fuzzy(x):.3f}') #final result
```

$$\mu_A(x) = \begin{cases} 0 & \text{if } x \leq a \\ \frac{x-a}{b-a} & \text{if } a \leq x \leq b \\ \frac{c-x}{c-b} & \text{if } b \leq x \leq c \\ 0 & \text{if } x \geq c \end{cases}$$

Output:

```
[np.float64(0.3333333333333333), np.float64(0.2857142857142857), 0]
[np.float64(27.5), np.float64(42.5), np.float64(52.5)]
34.423
```

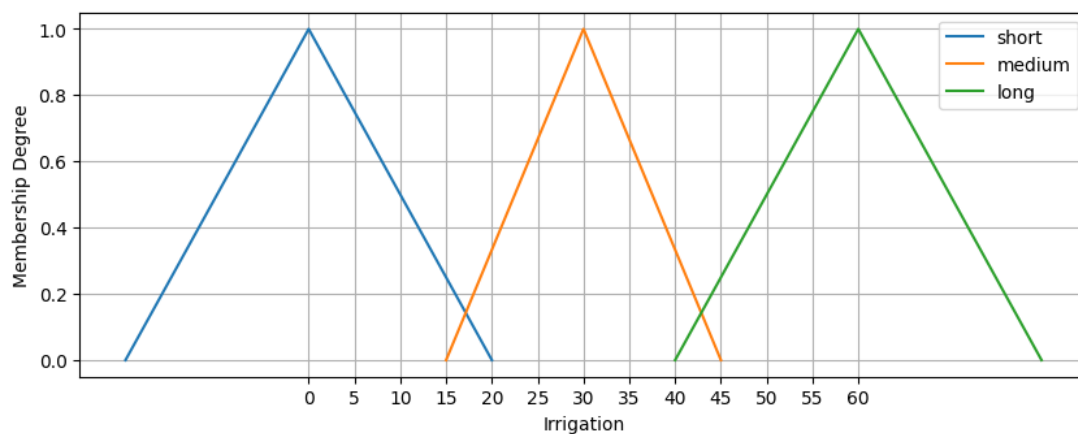
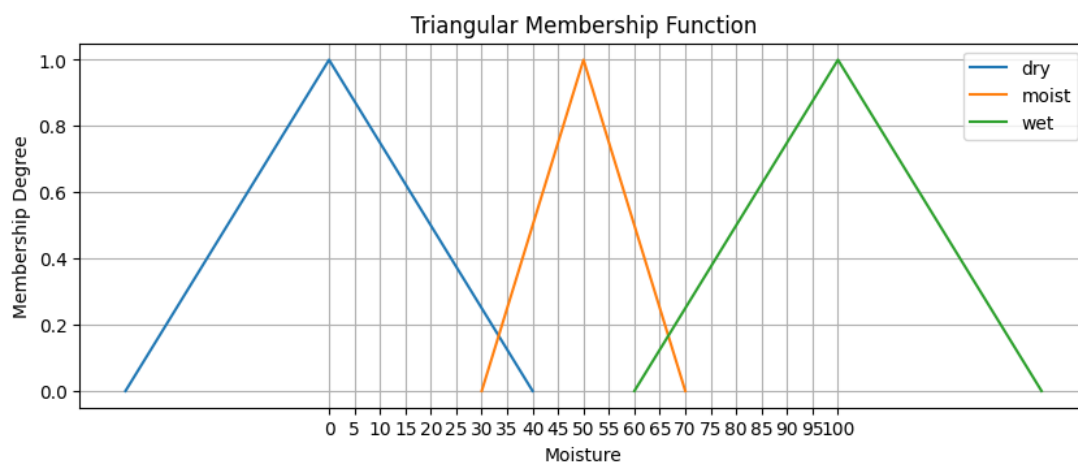
Problem-02: Soil's Moisture Level Controller

Input	Output
Moisture={ 'dry':[-40,40], 'moist':[30,70], 'wet':[60,140] }	Irrigation={ 'short':[-20,20], 'medium':[15,45], 'long':[40,80] }

#Rules:

1. If the soil is dry, the irrigation time will take long.
2. If the soil is moisture, the irrigation time will take medium.
3. If the soil is wet, the irrigation time will take short.

Membership Graph:



Source Code:

```
def rules(temp):
    if(temp=='dry'):
        return np.sum(Irrigation['long'])/2
    elif(temp=='moist'):
        return np.sum(Irrigation['medium'])/2
    elif(temp=='wet'):
        return np.sum(Irrigation['short'])/2
```

```
def membership_degree(x,s):
    a=Moisture[s][0]
    b=np.mean(Moisture[s])
    c=Moisture[s][1]
    if x>=c:
        return 0
    if x>b:
        return (c-x)/(c-b)
    if x>a:
        return (x-a)/(b-a)
    else:
        return 0
```

```
def Fuzzy(x):
    membership_value=[]
    for i in Moisture:
        result=membership_degree(x,i)
        membership_value.append(result)
    print(membership_value)

    crisp_irrigation=[]
    for i in Moisture:
        crisp_irrigation.append(rules(i))
    print(crisp_irrigation)

    crisp_irrigation=np.array(crisp_irrigation)
    try:
        crisp_irrigation=crisp_irrigation*np.array(membership_value)
        crisp_irrigation=np.sum(crisp_irrigation)/np.sum(membership_value)
    except:
        crisp_irrigation=0

    return crisp_irrigation
```

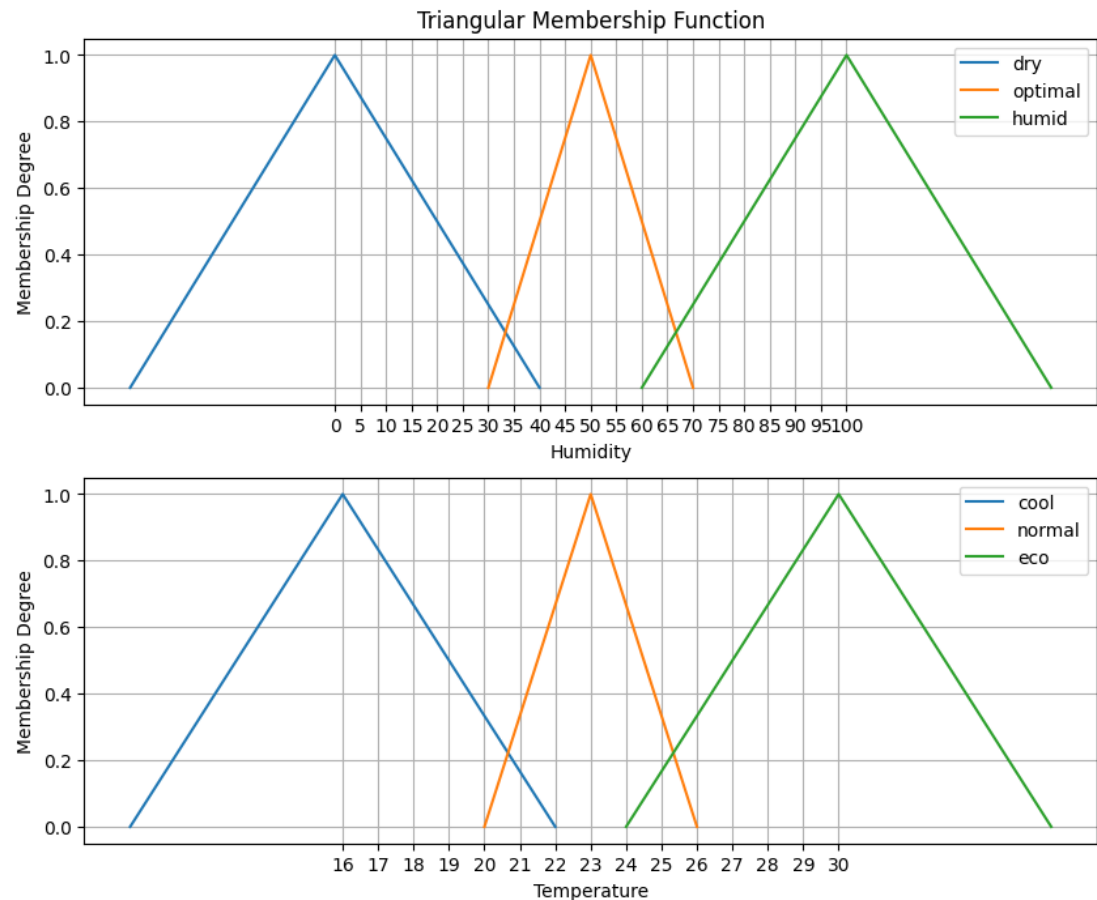
```
x=39
print(f" {Fuzzy(x):.3f} ") #final result
```

$$\mu_A(x) = \begin{cases} 0 & \text{if } x \leq a \\ \frac{x-a}{b-a} & \text{if } a \leq x \leq b \\ \frac{c-x}{c-b} & \text{if } b \leq x \leq c \\ 0 & \text{if } x \geq c \end{cases}$$

Output:

```
[np.float64(0.025), np.float64(0.45), 0]
[np.float64(60.0), np.float64(30.0), np.float64(0.0)]
31.579
```

Problem-03: AC Temperature Controller

Input	Output
humidity={ 'dry':[-40,40], 'optimal':[30,70], 'humid':[60,140] }	temperature={ 'cool':[10,22], 'normal':[20,26], 'eco':[24,36] }
#Rules: 1. If the Humidity is optimal, the temperature will set to normal range. 2. If the Humidity is humid, the temperature will set to eco-friendly range. 3. If the Humidity is dry, the temperature will set to cooling range.	
# Membership Graph: Triangular Membership Function  The top graph, titled 'Triangular Membership Function', plots the membership degree (y-axis, 0.0 to 1.0) against humidity (x-axis, 0 to 100). It shows three functions: 'dry' (blue) peaking at 0, 'optimal' (orange) peaking at 50, and 'humid' (green) peaking at 100. The bottom graph plots the membership degree (y-axis, 0.0 to 1.0) against temperature (x-axis, 16 to 30). It shows three functions: 'cool' (blue) peaking at 16, 'normal' (orange) peaking at 23, and 'eco' (green) peaking at 30.	

Source Code:

```
def rules(value):
    if(value=='optimal'):
        return np.sum(temperature['normal']/2
    elif(value=='humid'):
        return np.sum(temperature['eco']/2
    elif(value=='dry'):
        return np.sum(temperature['cool']/2

def membership_degree(x,s):
    a=humidity[s][0] # minimum value
    b=np.sum(humidity[s])/2 # mid value
    c=humidity[s][1] #max value
    if x>=c:
        return 0
    if x>b:
        return (c-x)/(c-b)
    if x>a:
        return (x-a)/(b-a)
    else:
        return 0

def Fuzzy(x):
    membership_value=[]
    for i,j in humidity.items():
        result=membership_degree(x,i) # take the membership value from given input
        membership_value.append(result)
    print(membership_value)

    crisp_temp=[]
    for i in humidity:
        crisp_temp.append(rules(i)) # storing the rule
    print(crisp_temp)

    crisp_temp=np.array(crisp_temp)
    try:
        crisp_temp=crisp_temp*np.array(membership_value) #multiplication
        crisp_temp=np.sum(crisp_temp)/np.sum(membership_value) # Divide
    except:
        crisp_temp=0
    return crisp_temp

x=35
print(f" {Fuzzy(x):.3f} ") #final result
```

$$\mu_A(x) = \begin{cases} 0 & \text{if } x \leq a \\ \frac{x-a}{b-a} & \text{if } a \leq x \leq b \\ \frac{c-x}{c-b} & \text{if } b \leq x \leq c \\ 0 & \text{if } x \geq c \end{cases}$$

Output:

```
[np.float64(0.125), np.float64(0.25), 0]
[np.float64(16.0), np.float64(23.0), np.float64(30.0)]
20.667
```

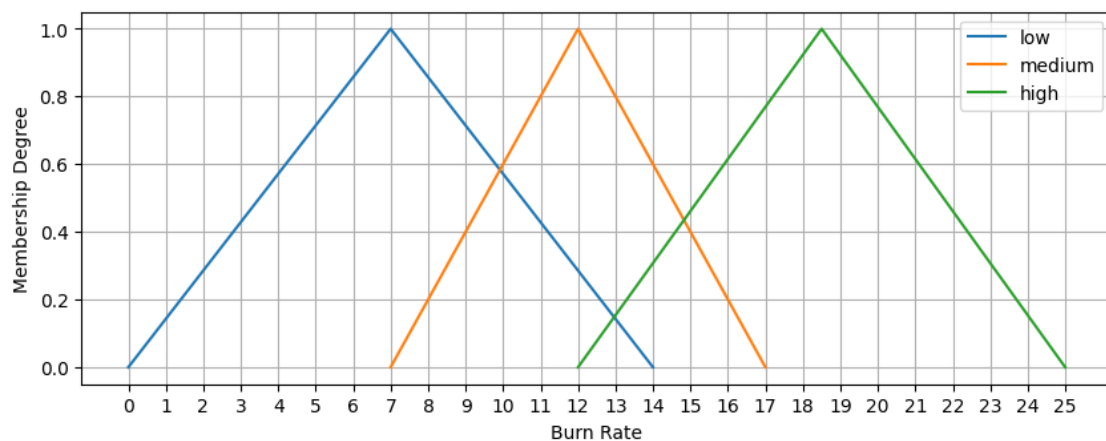
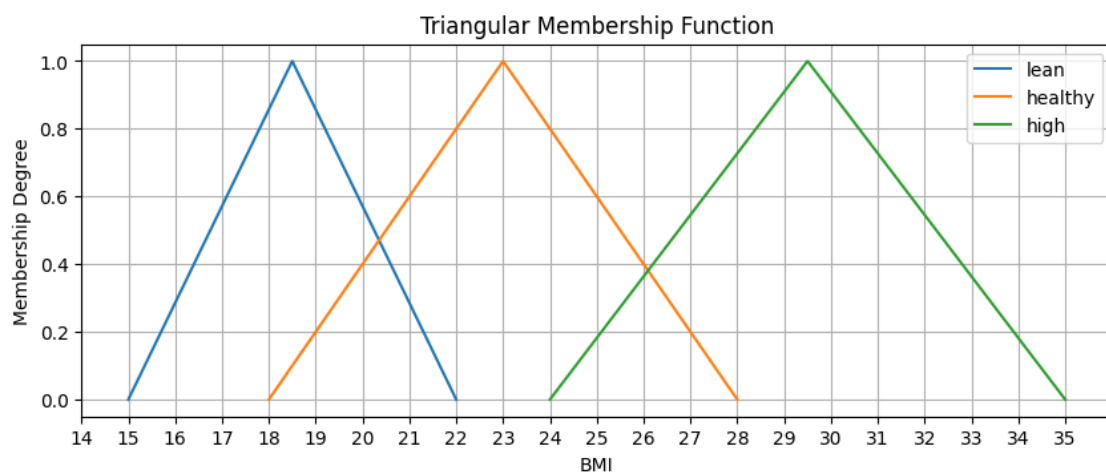
Problem-04: Daily Calorie Burn Rate Determination

Input	Output
bmi={ 'lean':[15,22], 'healthy':[18,28], 'high':[24,35] }	burn_rate={ 'low':[0,14], 'medium':[7,17], 'high':[12,25] }

#Rules:

1. If the BMI is in healthy range, the Burn Rate is medium.
2. If the BMI is in high range, the Burn Rate is high.
3. If the BMI is in lean range, the Burn Rate is low.

Membership Graph:



Source Code:

```
def rules(value):
    if(value=='healthy'):
        return np.sum(burn_rate['medium'])/2
    elif(value=='high'):
        return np.sum(burn_rate['high'])/2
    elif(value=='lean'):
        return np.sum(burn_rate['low'])/2

def membership_degree(x,s):
    a=bmi[s][0] # minimum value
    b=np.mean(bmi[s]) # mid value
    c=bmi[s][1] #max value
    if x>=c:
        return 0
    if x>b:
        return (c-x)/(c-b)
    if x>a:
        return (x-a)/(b-a)
    else:
        return 0

def Fuzzy(x):
    membership_value=[]
    for i,j in bmi.items():
        result=membership_degree(x,i) # take the membership value from given input
        membership_value.append(result)
    print(membership_value)

    crisp_burn=[]
    for i in bmi:
        crisp_burn.append(rules(i)) # storing the rule
    crisp_burn=np.array(crisp_burn)

    try:
        crisp_burn=crisp_burn*np.array(membership_value) #multiplication
        crisp_burn=np.sum(crisp_burn)/np.sum(membership_value) # Divide
    except:
        crisp_burn=0

    return crisp_burn

x=26
print(f'{Fuzzy(x):.3f}') #final result
```

$$\mu_A(x) = \begin{cases} 0 & \text{if } x \leq a \\ \frac{x-a}{b-a} & \text{if } a \leq x \leq b \\ \frac{c-x}{c-b} & \text{if } b \leq x \leq c \\ 0 & \text{if } x \geq c \end{cases}$$

Output:

```
[np.float64(0.025), np.float64(0.45), 0]  
[np.float64(60.0), np.float64(30.0), np.float64(0.0)]  
31.579
```

Conclusion:

In conclusion, Fuzzy Logic Systems provide a mathematical framework for modeling uncertainty and vagueness, allowing machines to make decisions based on ambiguous data like human intuition. By processing "degrees of truth" rather than binary absolutes, it enables the creation of control systems that are more robust, adaptable, and easier to implement for complex, non-linear problems. Key Applications are Consumer Electronics, Automotive Systems, Air Conditioning and Climate Control, Medical Diagnosis Systems, Pattern Recognition and Image Processing, Aerospace Control, Stock Market Prediction Systems